Accelerating Goals-Based Wealth Management via Neural Surrogate Look-Ahead State Approximations

Abstract

Meta-reinforcement learning (MetaRL) can produce near-optimal goals-based wealth-management strategies for a new investor without re-solving a dynamic program, but each decision requires two look-ahead *indicator* variables, g_{sim} and p_{sim}, that are computed by a forward simulation over 15 portfolios and 11 representative market shocks. In our profiling this simulation accounts for about 99% of single-query decision latency (9.7 ms of 9.8 ms). We replace it with a leakage-free *triplet surrogate*: a best-option classifier, a g_{sim} regressor and a p_{sim} regressor, each a $\sim$3.7k-parameter MLP, whose outputs are inserted into the state of the *unchanged* PPO actors, together with a simple wealth guardrail for out-of-distribution inputs. On a single CPU thread the indicator stage drops from 9.7 ms to 0.022 ms ($\approx 440\times$) and the full decision from 9.8 ms to 0.16 ms ($\approx 63\times$); the advantage shrinks to $14\times$ and $6\times$ at 10,000 batched states. On goal structures unseen in training the actors make the same decision as with exact indicators 94–96% of the time (goal) and select the identical portfolio index 90–91% of the time (95–96% within one index step). On a full time–wealth grid for held-out Test Case 1 the surrogate pipeline matches the original on 90.5% of portfolio decisions (96.3% within ± 1). In a 5,000-path Monte Carlo study of all 66 benchmark cases with common random numbers and group-held-out surrogates, the surrogate pipeline earns on average 98.5% of the original MetaRL's expected utility (median 99.9%, worst case 87.3%; 60 of 66 cases within 5%). We do not compare against dynamic programming; Section 7 lists what remains to be established.

Keywords: goals-based wealth management, meta-reinforcement learning, surrogate models, amortized inference, real-time advice.

1 Introduction

Goals-based wealth management (GBWM) asks how an investor should invest, and which of a sequence of financial goals to fund, so as to maximize expected goal utility over a multi-year horizon. Exact solutions use dynamic programming (DP) on a discretized wealth grid [2]; their cost grows exponentially with the number of state variables, and every new investor requires a new solve. Das et al. [1] instead pre-train a PPO [3] policy on thousands of GBWM problems (*MetaRL*); at deployment a new problem needs only forward passes of two small networks, and the paper reports strategies averaging about 97.8% of DP-optimal expected utility at more than $100\times$ lower cost.

A key ingredient of MetaRL is a pair of hand-engineered look-ahead *indicator* variables in the state: g_{sim}, a confidence score for taking the goal currently available, and p_{sim}, the normalized index of the portfolio that looks best. Both come from a deterministic forward simulation, and both are computed at every state the policy visits. For a robo-advisor answering one client at a time, this simulation, not the policy network, sets the latency.

Contributions.

1. **Profiling.** We localize the cost of a MetaRL decision: with Numba-compiled simulation code, indicator construction takes 9.7 ms against 0.13 ms for both actor networks (Section 6.3).
2. **A leakage-free triplet surrogate.** We replace the simulation by three small MLPs trained on *pre-clip, pre-filter* features, so that the training inputs cannot encode the label (Section 4), and add a wealth guardrail for inputs beyond the training range.
3. **Evaluation.** We report (i) group-held-out accuracy of several surrogate families, (ii) a feature ablation, (iii) latency from single queries to 10^4-state batches, and (iv) an end-to-end comparison on a full time–wealth grid of held-out Test Case 1, and (v) a Monte Carlo utility comparison over all 66 cases at four initial-wealth levels.

We state the boundary of the evidence: decision agreement and Monte Carlo utility relative to the exact-indicator pipeline are measured; comparison with DP is not (Section 7).

2 Background and Related Work

GBWM and DP. An investor with wealth W_t faces goals (t_g, C_g, u_g) with cost C_g and utility u_g, and may receive cash infusions I_t. At each year the investor chooses a portfolio $p \in \{0, \ldots, 14\}$ on a Markowitz efficient frontier [4] ordered from conservative to aggressive, with expected return μ_p and volatility σ_p, and decides whether to fund the goal(s) falling due. Wealth after spending s_t evolves as

$$W_{t+1} = (W_t - s_t)\exp\left[\left(\mu_p - \tfrac{1}{2}\sigma_p^2\right)h + \sigma_p\sqrt{h}Z_t\right] + I_{t+1}, \quad (1)$$

with $Z_t \sim \mathcal{N}(0,1)$. DP solves the resulting Bellman recursion on a (t, W) grid [2].

MetaRL. MetaRL [1] trains a *goal-taking* actor and a *portfolio* actor, each a feed-forward network (256–64–16–1, sigmoid

Table 1. Layout of the 27-dimensional MetaRL state.

Slot	Content
0	g_{sim} (goal call only)
1	t/T
2	p_{sim} (portfolio call only)
3, 4	$W/\text{PV}_{\text{pess}}$, W/PV_{opt} (pessimistic / optimistic goal PV)
5	inflation multiplier (constant 1 here)
6–12	$\text{GC}_{\min}/W$: 7 aggregated goal-cost buckets
13–19	G_u/U_{tot}: 7 aggregated goal utilities
20–26	$\text{GC}_{\max}/W$: 7 aggregated goal-cost buckets

output), by PPO across many problems whose goals, costs and utilities are randomized. The policy sees a 27-dimensional state that includes the two look-ahead indicators; the paper notes that state computation is expensive enough to warrant Numba compilation and parallelization.

Amortizing a look-ahead. Replacing an expensive deterministic computation by a learned approximation is a standard way to move cost offline. A related alternative is to distill the *final decisions* of a teacher into a student network [5]. We chose to approximate the indicators instead and keep the trained actors in the loop, because the actors are the part that was trained against the reward; Section 7 notes that we have not benchmarked the distillation alternative.

3 The Indicator Bottleneck

3.1 State vector

The state $\boldsymbol{S} \in \mathbb{R}^{27}$ has six leading features and three blocks of seven aggregated goal buckets (Table 1). Slot 0 holds g_{sim} and is filled only for the goal decision; slot 2 holds p_{sim} and is filled only for the portfolio decision. The remaining 25 slots are the *base features*; slot 5 (inflation multiplier) is constant in our experiments, leaving 24 informative inputs. Two post-processing steps in the environment matter later: (i) when $g_{\text{sim}} \geq 0.75$, the cost-to-wealth buckets (slots 6–12 and 20–26) are clipped to $[0, 0.5]$; (ii) every slot is clipped to $[-2, 2]$.

3.2 How the indicators are produced

Instead of Monte Carlo, the simulation uses $N = 11$ equal-probability representative shocks $z_i = \Phi^{-1}\big(\frac{i-1/2}{N}\big)$ and, for each of the $P = 15$ portfolios, holds that portfolio fixed over the remaining horizon, discounts future goal costs and infusions to time t, and funds goals in order of utility. Averaging over shocks gives

$$\mathbb{E}[U \mid p, a] = \tfrac{1}{11} \textstyle\sum_{i=1}^{11} U_{\text{tot}}(p, z_i, a), \qquad a \in \{\text{take}, \text{skip}\}. \quad (2)$$

The portfolio indicator is the normalized maximizer,

$$p_{\text{sim}} = \frac{p^\star}{P-1}, \qquad p^\star = \arg\max_p \ \mathbb{E}[U \mid p], \qquad (3)$$

so $p_{\text{sim}} = 0$ is the most conservative and $p_{\text{sim}} = 1$ the most aggressive portfolio. The goal indicator compares the best option with the runner-up: with $V(o) = u_o + \tilde{U}(W - c_o)$ the value of current-goal option o (cost c_o, utility u_o; "skip" has $c_o = 0$) and $\tilde{U}(w) = \max_p \mathbb{E}[\text{future utility} \mid w, p]$,

$$g_{\text{sim}} = \sigma\Big(\kappa \frac{V_{\text{best}} - V_{\text{next}}}{u_1}\Big), \qquad \kappa = 5, \qquad (4)$$

where u_1 is the utility of the first take option and σ is the logistic function (steepness as in the released code). The same routine also returns the *costs* of the best and next-best option; the goal actor's rounded output then selects which of the two is spent, so "action = 1" means "follow the best option" and a realized take/skip decision is $[\text{spend} > 0]$. The released implementation evaluates $\tilde{U}$ on a $\sim$250-point log-spaced wealth grid and interpolates, rather than looping over 165 explicit paths.

3.3 Profiling

Table 4 (Section 6.3) shows that, per decision, the simulation for g_{sim} and one call for p_{sim} cost $6.2 + 3.5 = 9.7$ ms, whereas both actor forward passes cost $2 \times 0.067 = 0.13$ ms: indicator construction is $\approx 98.6\%$ of the 9.8 ms decision. The cost is dominated by a fixed per-call overhead (grid and tensor construction) that is largely independent of how many states are processed together; this is why the bottleneck is acute for single queries and amortizes in large batches. (In an earlier measurement with Numba JIT disabled, state construction took 13.4 ms against 0.035 ms for an actor pass; we use the compiled figures throughout.)

4 Method: A Leakage-Free Triplet Surrogate

4.1 Base features without leakage

Both post-processing steps of Section 3 depend on the true g_{sim}: the filter changes the inputs of high-confidence rows, so a surrogate trained on filtered features could partly read its target from its own inputs. We therefore recompute the base features *before* the filter and the clip. Given the true indicators, re-applying the filter and the clip to these features reproduces the environment's state exactly (maximum absolute difference 0.0 on the grids we checked). Heavy-tailed ratio features ($W/\text{PV}_{\text{pess}}$, W/PV_{opt} and the cost buckets) are passed through $\log(1+x)$, and all inputs are standardized with training statistics.

4.2 The triplet

Let $\boldsymbol{x} = \phi(\boldsymbol{s}_{\text{base}}) \in \mathbb{R}^{24}$ be the transformed features. The surrogate consists of *three separate networks* f_b, f_g, f_p with no shared layers or weights. They have the same architecture, a 3-layer MLP $f_\theta(\boldsymbol{x}) = \sigma\big(\boldsymbol{w}_3^\top \text{ReLU}(\boldsymbol{W}_2 \text{ReLU}(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2) + b_3\big)$ with layout 24→64→32→1, but each has its own

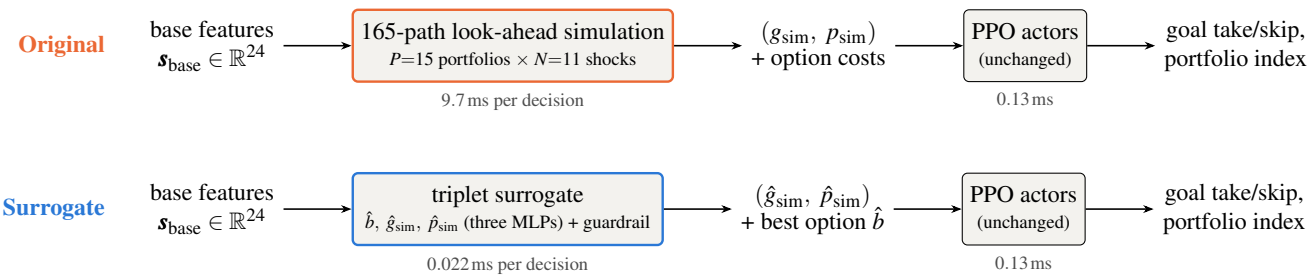


Figure 1. Original and surrogate decision pipelines. The surrogate replaces only the indicator stage; the trained goal and portfolio actors are used unchanged. Latencies are single-state CPU medians (Table 4).

parameters $\theta_b, \theta_g, \theta_p$ and is trained independently on its own target:

$$\hat{b} = f_{\theta_b}(\boldsymbol{x}) \approx \Pr[\text{best option is a take}], \quad \mathscr{L}_b = \text{BCE}(b, \hat{b}), \tag{5}$$

$$\hat{g}_{\text{sim}} = f_{\theta_g}(\boldsymbol{x}) \approx g_{\text{sim}}, \quad \mathscr{L}_g = (g_{\text{sim}} - \hat{g}_{\text{sim}})^2, \tag{6}$$

$$\hat{p}_{\text{sim}} = f_{\theta_p}(\boldsymbol{x}) \approx p_{\text{sim}}, \quad \mathscr{L}_p = (p_{\text{sim}} - \hat{p}_{\text{sim}})^2. \tag{7}$$

$\hat{b}$ is needed because the identity of the best option is a by-product of the simulation, and the actor's output only selects between the best and the next-best option. In the single-take-option steps of the benchmark the next-best option is *never* a take (0 of 277,903 affordable training rows), so a goal is funded if and only if the actor follows a best option that is a take. This also covers the common case where the simulation finds no take worthwhile and returns both option costs as zero. (An earlier version of our evaluation instead assumed that, when the best option is "skip", the next-best option is the take; this silently funded goals the original pipeline would not fund and cut utility on 23 of 66 cases, by up to 90%. We report only the corrected rule.) Each network has 3,713 parameters (3,616 multiply–accumulates). The portfolio network is queried at the *post-decision* wealth of the pipeline's own goal decision, as the original portfolio actor is.

4.3 Inference and guardrail

Algorithm 1 gives the surrogate decision at one time step. Two rules are structural and need no learning: a goal that cannot be afforded ($W < c$) is never funded, and when no goals remain after the current step the exact code returns $p_{\text{sim}} = 0$, which we reproduce. For out-of-distribution wealth we apply two guardrails: all standardized inputs are clamped to their training range, and

$$W > 2\sum_{\tau \geq t} C_\tau \implies \hat{g}_{\text{sim}} \leftarrow 1,\ \hat{b} \leftarrow 1, \tag{8}$$

i.e. an investor with more than twice their remaining (undiscounted) goal cost always takes an affordable goal. The threshold is a design choice. In our experiments it never changed the outcome (Section 6.5): input clamping alone already pins the surrogate near the correct behavior at high wealth.

Algorithm 1 Surrogate decision at step t with wealth W

$\boldsymbol{x} \leftarrow \phi(\text{BASEFEATURES}(t, W))$, clamp $\boldsymbol{x}$ to training range
$(\hat{g}_{\text{sim}}, \hat{b}) \leftarrow (f_g(\boldsymbol{x}), f_b(\boldsymbol{x}))$
if $W > 2\sum_{\tau \geq t} C_\tau$ **then** $\hat{g}_{\text{sim}} \leftarrow 1,\ \hat{b} \leftarrow 1$
end if
$\hat{\boldsymbol{S}} \leftarrow \text{FILTERANDCLIP}(\boldsymbol{x}, \hat{g}_{\text{sim}});\ a_g \leftarrow \lfloor \pi_{\text{goal}}(\hat{\boldsymbol{S}}) \rceil$
$s \leftarrow c \cdot a_g \cdot [\hat{b} \geq \frac{1}{2}] \cdot [W \geq c]$
$\boldsymbol{x}' \leftarrow \phi(\text{BASEFEATURES}(t, W - s));\ \hat{p}_{\text{sim}} \leftarrow f_p(\boldsymbol{x}')$ (0 if no goals remain)
$a_p \leftarrow \lfloor 14\, \pi_{\text{port}}(\text{CLIP}(\boldsymbol{x}', \hat{p}_{\text{sim}})) \rceil$
return take$= [s > 0]$, portfolio index a_p

5 Experimental Setup

Problems and teacher. The 66 benchmark scenarios of the released code: cases 1–33 without and cases 34–66 with cash infusions, which share goal structures pairwise (k and k+33). The teacher is the shipped pair of PPO actors (single model seed) with exact indicators and a P=15 efficient frontier; inflation is off.

Two studies. (1) A *development study* with 5-fold cross-validation grouped by goal structure (the held-out fold contains structures never seen in training), comparing ridge regression, a 64–32 tanh MLP and histogram gradient boosting [8], with a feature ablation and a latency benchmark. Its dataset was built from the environment's *post*-filter state, so its accuracy figures may be optimistic. (2) A *leakage-free study* using the pre-clip, pre-filter pipeline of Section 4: 328,000 goal rows and 1,071,000 portfolio rows from all 66 cases (wealth log-uniform over $[0.05, 3] \times \text{PV}_{\text{pess}}$, 500 draws per step), with Test Case 1 and its twin Case 34 *held out entirely*. Each network is trained with Adam [6] for 25 epochs (batch 4096, cosine schedule, MSE or BCE as above); fitting takes 12, 7 and 27 s for $\hat{g}_{\text{sim}}$, $\hat{b}$, $\hat{p}_{\text{sim}}$ on one CPU core.

Monte Carlo study. For each of the 66 cases we simulate 5,000 wealth paths under eq. (1) with shocks $Z_{p,t}$ shared by all policies (common random numbers), the released initial wealth multiplied by a ratio ρ, and report mean realized goal utility. We compare (a) the original MetaRL (exact indicators), (b) the surrogate pipeline with the guardrail, (c) the same without the guardrail, and (d) a naive heuristic (fund every affordable goal, always use portfolio index 7). Surrogates

Table 2. Development study: 5-fold group-held-out accuracy. "Actor" = agreement when the actor receives the predicted indicator; "Round" = agreement from rounding the prediction directly. Portfolio columns compare actor-chosen indices.

	Goal (g_{sim})			Portfolio (p_{sim})		
Model	R^2	Actor	Round	R^2	Exact	± 1
Ridge	0.825	87.9	86.8	0.849	64.0	79.0
MLP 64–32	0.914	93.8	85.6	0.946	90.4	94.8
Hist. GB	**0.961**	**95.5**	**88.1**	**0.968**	**91.4**	**96.4**

are scored by 3-fold *group* cross-validation: case k is evaluated with the fold whose training data excludes goal structure $(k-1) \bmod 33$ and 10 others. Our simulator reproduces the released `RLMC_pathbased` reward exactly (Cases 1 and 24, same seed). The default $\rho = 0.58$ is that of the released script; $\rho \in \{0.25, 3, 10\}$ are stress levels.

Metrics. *Goal agreement*: the realized take/skip decision is identical to the teacher's. *Portfolio exact* and *within-*± 1: the chosen index $\lfloor 14a \rceil$ equals, or is within one of, the teacher's. In the development study, agreement is measured by feeding the predicted indicator to the actor and comparing with the actor's output under the true indicator.

Hardware and timing. One CPU thread, Numba [7]-compiled exact simulation, PyTorch actors, a NumPy forward pass for the surrogate MLP, medians over four (case, step) configurations (cases 24, 22, 33, 66). The surrogate used for timing has untrained weights of the deployed architecture (26–64–32–1, tanh), which does not change its latency.

6 Results

6.1 Surrogate accuracy on unseen goal structures

Table 2 reports the development study. A linear model is clearly insufficient; the MLP and gradient boosting are close, with boosting about 1–2 points better. Actor agreement is markedly higher than agreement obtained by rounding the surrogate itself (93.8% vs. 85.6% for the MLP): errors concentrate near the take/skip boundary, where the actor is insensitive to small changes in the indicator. This supports keeping the actors in the loop.

6.2 Which features matter

Table 3 shows that g_{sim} is almost a function of two wealth ratios: $W/\text{PV}_{\text{pess}}$ and W/PV_{opt} alone give $R^2 = 0.950$ against 0.961 with all features. This makes a closed-form rule for goal-taking plausible, although we have not derived one; the table is a fitted model, not a formula. p_{sim} behaves differently: the two ratios give 0.835, the time fraction lifts this to 0.923, and the full goal structure is needed for 0.968. The optimal portfolio depends on *when* and *how much* future goals cost, which two ratios cannot encode.

Table 3. Feature ablation (gradient boosting, 5-fold group CV).

	g_{sim}		p_{sim}	
Inputs	R^2	MAE	R^2	MAE
A: $W/\text{PV}_{\text{pess}}$, W/PV_{opt}	0.950	0.023	0.835	0.062
B: A $+ t/T$	0.951	0.021	0.923	0.034
C: B + current goal	0.954	0.019	0.922	0.034
D: all 24 features	**0.961**	**0.018**	**0.968**	**0.017**

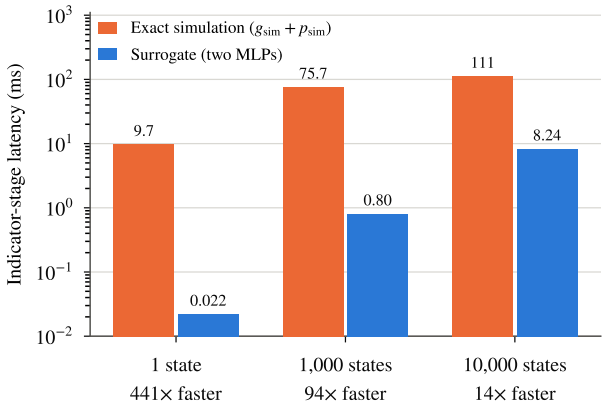


Figure 2. Indicator-stage latency (one g_{sim} plus one p_{sim} call vs. two surrogate MLP passes), log scale, mean over four case/step configurations. Bar labels give milliseconds.

6.3 Latency

Figure 2 and Table 4 give the cost of the indicator stage (one g_{sim} call and one p_{sim} call versus two surrogate passes) and of the full decision (indicators plus both actors). For a single state the indicator stage is 441× faster (0.022 ms vs. 9.7 ms; the g_{sim} stage alone ranges from 404× to 803× over the four configurations) and the full decision 63× faster. The advantage shrinks with batch size, because the exact simulation amortizes its fixed overhead: 94× and 14× for the indicator stage at 10^3 and 10^4 states, and 33× and 6× for the full decision. After the swap the two torch actor passes (0.134 ms) account for 86% of the single-state decision time, so exporting the actors to a lighter runtime is the next obvious step; we did not measure it. Gradient boosting is 13–19× faster than the exact g_{sim} for $\leq 10^3$ states but only 3.6× at 10^4, and always slower than the MLP, so the 1–2 accuracy points it gains in Table 2 cost an order of magnitude in latency.

6.4 End-to-end policy fidelity on held-out Test Case 1

We evaluate both pipelines on a grid of 11 decision steps × 240 log-spaced wealth values in $[10, 600]$ (2,640 states). The surrogates never saw Case 1 or Case 34 in training. On the held-out rows themselves the networks reach $R^2 = 0.991$ ($\hat{g}_{\text{sim}}$, MAE 0.006) and $R^2 = 0.989$ ($\hat{p}_{\text{sim}}$, MAE 0.016, 87.3% exact

Table 4. Component latency in ms (single CPU thread; mean of per-configuration medians). Pipeline = $g_{\text{sim}} + p_{\text{sim}} + 2\times$actor (exact) or $2\times$MLP$+2\times$actor (surrogate).

States	g_{sim}	p_{sim}	Actor	MLP	Pipeline exact	Pipeline surr.
1	6.17	3.51	0.067	0.011	9.81	0.155
1,000	39.6	36.1	0.77	0.40	77.3	2.34
10,000	61.9	49.5	6.05	4.12	123.5	20.3
End-to-end speedup					63× / 33× / 6×	

Table 5. Test Case 1, original vs. surrogate pipeline on the full time–wealth grid.

Quantity	States	Result
Goal take/skip agreement	240	100%
g_{sim} MAE	240	0.0074
Portfolio index, exact	2,640	90.5%
Portfolio index, within ± 1	2,640	96.3%
mean $\lvert\Delta\text{index}\rvert$; max	2,640	0.147; 4
p_{sim} MAE	2,640	0.0159
Portfolio exact, inside training range	2,570	90.3%
Portfolio exact, outside training range	70	100%

index).

Table 5 and Figure 3 show that the surrogate pipeline reproduces the teacher's policy surface. Portfolio agreement is lowest at early steps (exact match 87–89% for $t \leq 1$) and rises with t (94% at $t = 8$, 100% at $t = 10$); the mismatches are confined to the wealth band where the teacher switches from aggressive to conservative portfolios, and the worst miss is four index steps.

Two caveats limit what this experiment shows. *First*, Case 1 contains a single goal, located at the *final* decision step with no goals after it. At that step the exact code does not run the look-ahead at all: it returns $g_{\text{sim}} = 1$ if $W \geq 150$ and 0 otherwise. The 100% goal agreement therefore only shows that the surrogate learned an affordability rule, and the goal panel is a one-step profile rather than a time–wealth surface; the best-option classifier is likewise trivially accurate here (every affordable state is a take). The portfolio surface is the informative part. *Second*, only 70 of the 2,640 grid states lie outside the training range of $W/\text{PV}_{\text{pess}} \in [0.05, 3]$, and the surrogate matched the teacher on all of them, so this grid exercises the guardrail only lightly.

6.5 Monte Carlo expected utility over 66 cases

Table 6 summarizes the utility study. At the default wealth ($\rho = 0.58$) the surrogate pipeline earns on average 98.5% of the original's mean utility (median 99.9%); 48 of 66 cases are within 1% and 60 within 5%. Six cases fall between 87% and 90% (cases 11, 15, 28, 44, 48, 61), and the two next-worst are at 95.6% (10, 43). Because the policies share shocks, the paired difference is precise: 25 cases differ significantly from the original ($|z| > 3$) in the negative direction and 6 in the positive direction, i.e. the surrogate is usually slightly worse

Table 6. Monte Carlo utility, surrogate pipeline as % of original MetaRL mean utility (5,000 paths, common random numbers, group-held-out surrogates). Guard on; the guard-off column is identical in every row.

ρ	Mean	Median	Min	≥95%	≥99%	Naive
0.25	98.2	99.8	84.7	56	46	22.8
0.58	98.5	99.9	87.3	60	48	40.0
3	99.8	100.0	97.2	66	60	78.7
10	100.0	100.0	99.0	66	66	92.9

but occasionally better, and the sizes are small except in the six cases above. The naive heuristic reaches only 40% on average, so the surrogate's score is not an artifact of easy cases. Figure 4 shows the full distribution.

At the stress levels the picture is consistent: the average is 98.2% at $\rho = 0.25$ (worst 84.7%, case 28), 99.8% at $\rho = 3$ and 99.97% at $\rho = 10$, where every case is above 99% and the naive heuristic also does well. Low-wealth investors are thus the harder regime. The wealth guardrail did not change mean utility in any of the 264 case–wealth combinations.

Compute time of the full Monte Carlo. We re-ran all 66 cases with 10,000 paths per case at $\rho = 0.58$ on one CPU thread, timing each policy end to end (JIT compilation excluded). The original pipeline took 236 s in total and the surrogate pipeline 96 s, a speedup of $2.5\times$ (excluding Case 1, whose surrogate timing absorbed a one-off compilation of the feature code: 235 s vs. 95 s, $2.5\times$; per-case median $2.4\times$, range 1.5–$4.6\times$; mean utility 98.5% of the original, as before). The gain is far smaller than the $63\times$ single-query figure because a Monte Carlo evaluates all 10^4 paths in one batch, where the exact simulation amortizes its fixed overhead (cf. $6\times$ for the full decision at 10^4 states in Table 4), and because the surrogate pipeline's remaining time is spent in unoptimized feature construction and the two actor networks, which we did not tune. The surrogate's advantage is therefore for interactive, per-client use rather than for bulk simulation.

7 Discussion and Limitations

What the evidence supports. Replacing the indicator simulation by three small MLPs removes ≈ 98.6% of single-decision latency and preserves the teacher's decisions on 94–96% of goal states and 90–91% of portfolio states on unseen goal structures, with nearly all portfolio disagreements within one index step. On held-out Test Case 1 the surrogate pipeline reproduces the teacher's portfolio surface at 90.5% exact agreement. In a 66-case Monte Carlo study the surrogate pipeline retains 98.5% of the original's mean utility on average, with a worst case of 87.3%.

What it does not yet support.

- **No DP comparison.** The released package does not include the DP solver, so utility is measured relative to the *original*

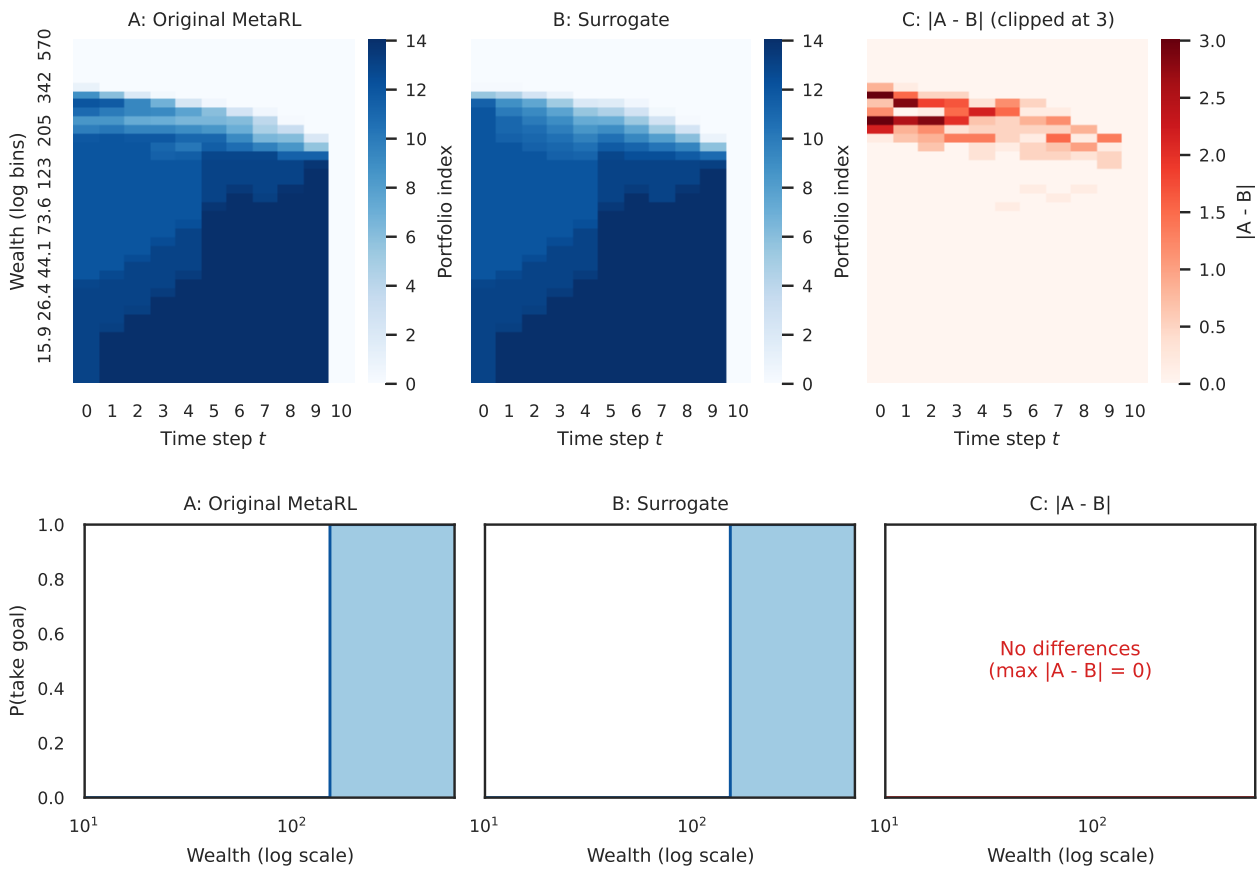


Figure 3. Test Case 1 policy surfaces. **Top:** portfolio index (0 = conservative, 14 = aggressive) over time and wealth for (A) the original MetaRL, (B) the surrogate pipeline, and (C) their absolute difference, clipped at 3 index steps. **Bottom:** probability of taking the goal at the only step that has one ($t = 10$). The surrogate reproduces the surface; differences lie in a narrow band along the aggressive-to-conservative transition at early steps.

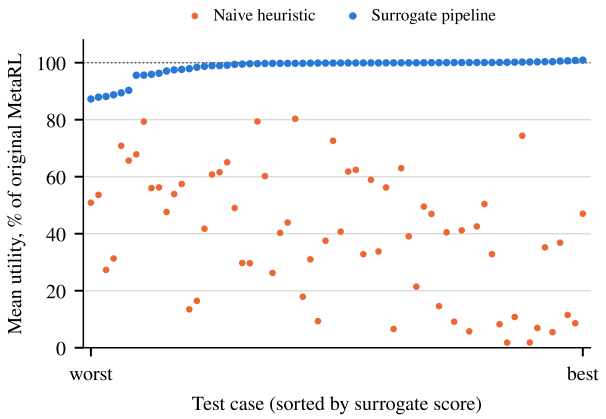


Figure 4. Mean realized utility per case at $\rho = 0.58$ as a percentage of the original MetaRL (66 cases sorted by surrogate score).

MetaRL, not to the optimum. We make no statement about DP or about the 97.8% reported for MetaRL [1] beyond noting that our numbers are conditional on that policy.

- **Weak cases.** Six cases (11, 15, 28, 44, 48, 61) retain only 87–90% of the original utility. We have not diagnosed why these cases are harder; they differ in goal structure and in the original's mean utility (0.6 to 6.3), so no single explanation is evident.
- **Guardrail.** The wealth guardrail of Equation (8) never changed mean utility, even at $\rho = 10$. Input clamping already handles high wealth, so we cannot claim a protective effect; the failure that motivated the guardrail (a high-wealth case in a preliminary run) is not reproduced under the corrected goal rule and was probably caused by the earlier rule error described in Section 4.
- **Grid study breadth.** The time–wealth grid study covers one case with a degenerate goal step; breadth comes from the Monte Carlo study, which does not report step-level agreement.
- **Leakage in the development study.** Tables 2 and 3 come from features taken after the environment's filter and clip. The held-out results of the leakage-free pipeline are consistent with them (87.3% exact portfolio index on the held-out rows against 90.4% for the comparable MLP), but the two studies differ in sampling and cases, so we do not treat them as a controlled comparison.
- **Single teacher.** All results use one actor seed; the original work ensembles models. Decision agreement is relative to

this teacher, which is not itself optimal.

- **Timing scope.** Latencies are single-thread CPU medians on four configurations; the surrogate was timed with untrained weights of the deployed shape; the exact simulation was timed after JIT warm-up. We did not time the actors under an optimized runtime.
- **Alternatives not benchmarked.** We did not train a direct teacher–student distillation model [5], a closed-form rule for g_{sim}, a cached lookup table, or a policy trained with annealed indicators. We therefore make no claim about their relative merit; the case for the modular design rests on keeping the trained actors and on the actor-agreement result of Table 2, not on a head-to-head comparison.

8 Conclusion and Future Work

The look-ahead simulation that builds the MetaRL state can be replaced, at single-query latency, by three small neural networks whose outputs the unchanged actors consume, cutting a decision from 9.8 ms to 0.16 ms while reproducing the teacher's decisions in the large majority of states, including on a goal structure held out of training. In 66-case Monte Carlo simulation this retains 98.5% of the original's mean utility on average. The advantage is largest for one-client-at-a-time use and shrinks to 6× at 10^4 batched states.

The next steps are the ones the evidence still lacks: (i) comparison against DP (not in the released package) and against several actor seeds; (ii) diagnosing and repairing the six weakest cases; (iii) an actor ensemble; (iv) exporting the actors to remove the remaining 0.13 ms; (v) a closed-form or tabulated g_{sim}, given $R^2 = 0.95$ from two wealth ratios; and (vi) uncertainty-triggered fallback to the exact simulation when surrogate disagreement is high, which would bound the error while keeping the speed.

A Reproducibility

Environment. Python 3.10, `numpy<2`, `numba==0.60.0`, PyTorch, scikit-learn, pandas, matplotlib/seaborn. (Python 3.12 with NumPy 2 breaks the released simulation code.)

Pipeline (leakage-free study). `build_dataset_c1.py` (pre-clip dataset, ≈160 s after a one-off Numba compile of ≈90 s) → `train_surrogates.py` (holds out cases 1 and 34) → `eval_case.py 1` (writes `eval_case1.csv` and metrics) → `plot_policy_heatmaps.py -paper` (the policy figures). `c1lib.py` implements the pre-clip features and verifies that filtering and clipping them with the true indicators reproduces the environment state. The development study used `build_dataset.py`, `exp1_surrogate.py` and `exp2_timing_ablation.py`.

Hyperparameters. MLP 24→64→32→1, ReLU, sigmoid output; Adam, learning rate 3×10^{-3}, cosine decay, 25 epochs, batch 4096, seed 0. The development-study MLP has 26 inputs (24 features plus $\log W/\text{PV}_{\text{pess}}$ and $\log W/\text{PV}_{\text{opt}}$), tanh activations, 3,841 parameters and was trained for 40 epochs.

B Held-out surrogate quality (leakage-free study)

Table 7. Surrogates evaluated on the held-out rows of cases 1 and 34.

Network	Train rows	Test rows	Metric	Value
$\hat{g}_{\text{sim}}$	327,000	1,000	R^2 / MAE	0.991 / 0.0064
$\hat{b}$	277,633	270	accuracy	100% (trivial)
$\hat{p}_{\text{sim}}$	1,059,000	12,000	R^2 / MAE	0.989 / 0.0156
			exact / ±1 index	87.3% / 95.4%

References

[1] S. R. Das, H. Khadilkar, S. Mittal, D. Ostrov, D. Srivastav, and H. Wang, "A meta reinforcement learning approach to goals-based wealth management," arXiv:2605.02300, 2026.

[2] S. R. Das, D. Ostrov, A. Radhakrishnan, and D. Srivastav, "Dynamic optimization for multi-goals wealth management," *Journal of Banking & Finance*, vol. 140, 2022, doi:10.1016/j.jbankfin.2021.106192.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[4] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[5] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, 2014.

[7] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC*, 2015.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.